

Phase I : Local CLI Shell Implementation

COSC 354 : Operating Systems

Spring 2026

by

Abi Ali Murtaza Ghadiyali 100066653
Muhammad Awais Afzal Mirza 100066617

Table of Contents:

Introduction:	2
General idea of the Program:.....	2
Constants:	2
Functions overview:	3
parse_line()	3
parse_command()	3
execute_builtin()	4
execute_single().....	4
execute_pipe()	5
main()	7
Limitations:	8
Conclusion:.....	8
Code:.....	9

Introduction:

In this phase, a simple Linux-like shell was implemented in C. The shell reads user input, parses commands, and executes them using system calls such as `fork()`, `execvp()`, and `pipe()`.

It supports:

- execution of Linux commands
- built-in commands (`cd`, `clear`, `exit`)
- pipelines with up to 3 pipes

General idea of the Program:

The shell follows this workflow:

1. Display prompt
2. Read input
3. Split commands using `|`
4. Split each command into arguments
5. Execute:
 - single command → `execute_single()`
 - multiple commands → `execute_pipe()`

Constants:

```
#define MAX_LINE 1024
#define MAX_CMDS 5
#define MAX_ARGS 20
```

- `MAX_LINE`: input size

- MAX_CMDS: max piped commands
- MAX_ARGS: max arguments per command

Functions overview:

parse_line()

Purpose: Split input into commands using |

- Stores each command in cmds[]
- Returns number of commands

```
int parse_line(char *line, char **cmds) {  
    int count = 0;  
    char *token = strtok(line, "|");  
  
    while (token != NULL && count < MAX_CMDS) {  
        cmds[count++] = token;  
        token = strtok(NULL, "|");  
    }  
  
    return count;  
}
```

parse_command()

Purpose: Split a command into arguments

- Uses spaces/tabs as delimiters
- Prepares array for execvp()

```
// Parse a command into arguments separated by spaces
int parse_command(char *cmd, char **args) {
    int count = 0;
    char *token = strtok(cmd, " \\t\\n");

    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \\t\\n");
    }
    args[count] = NULL;

    return count;
}
```

execute_builtin()

Purpose: Handles internal built in commands

- Supported:
 - exit → terminate shell
 - cd → change directory
 - clear → clear screen
- Returns:
 - 1 → executed
 - 0 → not built-in

```
// Execute built-in commands
int execute_builtin(char **args) {
    if (strcmp(args[0], "exit") == 0) {
        exit(0);
    } else if (strcmp(args[0], "cd") == 0) {
        if (args[1] == NULL) {
            fprintf(stderr, "shell: cd: no argument provided\n");
        } else {
            if (chdir(args[1]) != 0) {
                perror("shell: cd");
            }
        }
        return 1; // Built-in executed
    } else if (strcmp(args[0], "clear") == 0) {
        printf("\033[H\033[J");
        fflush(stdout);
        return 1; // Built-in executed
    }
    return 0; // Not a built-in
}
```

execute_single()

Purpose: Execute one command

Steps:

1. Checks built-in
2. fork() child
3. execvp() in child
4. wait() in parent

```
// Execute a single command
void execute_single(char **args) {
    if (execute_builtin(args)) {
        return;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("shell: fork failed");
        return;
    }

    if (pid == 0) {
        // Child process
        execvp(args[0], args);
        perror("shell: command not found");
        exit(1);
    } else {
        // Parent process
        wait(NULL);
    }
}
```

execute_pipe()

Purpose: Execute piped commands

- Steps:
 1. Create pipes (n-1)
 2. Fork for each command
 3. Use dup2() for redirection
 4. Execute using execvp()
 5. Parent waits for all children
- Supports:
 - 1 pipe → cmd1 | cmd2
 - 2 pipes → cmd1 | cmd2 | cmd3
 - 3 pipes → cmd1 | cmd2 | cmd3 | cmd4

```

// Execute piped commands
void execute_pipe(char **cmd_strings, int num_cmds) {
    int pipes[MAX_CMDS - 1][2];
    pid_t pids[MAX_CMDS];

    // Create all necessary pipes
    for (int i = 0; i < num_cmds - 1; i++) {
        if (pipe(pipes[i]) < 0) {
            perror("shell: pipe failed");
            return;
        }
    }

    // Fork and execute each command
    for (int i = 0; i < num_cmds; i++) {
        char *args[MAX_ARGS];
        parse_command(cmd_strings[i], args);

        if (args[0] == NULL) continue;

        pids[i] = fork();
        if (pids[i] < 0) {
            perror("shell: fork failed");
            return;
        }

        if (pids[i] == 0) {
            // Child process

            // Set up input from previous pipe
            if (i > 0) {
                dup2(pipes[i - 1][0], STDIN_FILENO);
            }

            // Set up output to next pipe
            if (i < num_cmds - 1) {
                dup2(pipes[i][1], STDOUT_FILENO);
            }

```

```

        // Close all pipe ends
        for (int j = 0; j < num_cmds - 1; j++) {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        // Execute built-in or external command
        if (execute_builtin(args)) {
            exit(0);
        }

        execvp(args[0], args);
        perror("shell: command not found");
        exit(1);
    }
}

// Parent: close all pipe ends
for (int i = 0; i < num_cmds - 1; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
}

// Wait for all children
for (int i = 0; i < num_cmds; i++) {
    wait(NULL);
}
}

```

main()

Purpose: Controls the main loop that runs forever until exited

- Reads input
- Parses commands
- Chooses execution type
- Runs continuously until exit

```
int main() {
    char line[MAX_LINE];
    char *cmds[MAX_CMDS];

    while (1) {
        // Print prompt
        printf("shell> ");
        fflush(stdout);

        // Read input
        if (fgets(line, sizeof(line), stdin) == NULL) {
            printf("\n");
            break;
        }

        // Skip empty lines
        if (line[0] == '\n' || line[0] == '\0') {
            continue;
        }

        // Parse line into commands
        int num_cmds = parse_line(line, cmds);

        if (num_cmds == 0) {
            continue;
        }

        // Execute
        if (num_cmds == 1) {
            char *args[MAX_ARGS];
            parse_command(cmds[0], args);
            if (args[0] != NULL) {
                execute_single(args);
            }
        } else {
            execute_pipe(cmds, num_cmds);
        }
    }

    return 0;
}
```

General flow of operations:

- `fork()` → create process
- `execvp()` → execute command
- `wait()` → synchronize processes
- `pipe()` → inter-process communication
- `dup2()` → redirect input/output
- `chdir()` → change directory

Limitations:

- no I/O redirection (`>`, `<`)
- no background execution (`&`)
- limited error handling
- basic parsing only

Conclusion:

The shell successfully demonstrates process creation, command execution, and inter-process communication. It provides a simplified understanding of how a Linux shell works.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_LINE 1024
#define MAX_CMDS 5
#define MAX_ARGS 20

// Parse a line into commands separated by '|'
int parse_line(char *line, char **cmds) {
    int count = 0;
    char *token = strtok(line, "|");

    while (token != NULL && count < MAX_CMDS) {
        cmds[count++] = token;
        token = strtok(NULL, "|");
    }

    return count;
}

// Parse a command into arguments separated by spaces
int parse_command(char *cmd, char **args) {
    int count = 0;
    char *token = strtok(cmd, " \\t\\n");

    while (token != NULL && count < MAX_ARGS - 1) {
        args[count++] = token;
        token = strtok(NULL, " \\t\\n");
    }
    args[count] = NULL;

    return count;
}

// Execute built-in commands
int execute_builtin(char **args) {
    if (strcmp(args[0], "exit") == 0) {
        exit(0);
    } else if (strcmp(args[0], "cd") == 0) {
```

```

    if (args[1] == NULL) {
        fprintf(stderr, "shell: cd: no argument provided\n");
    } else {
        if (chdir(args[1]) != 0) {
            perror("shell: cd");
        }
    }
    return 1; // Built-in executed
} else if (strcmp(args[0], "clear") == 0) {
    printf("\033[H\033[J");
    fflush(stdout);
    return 1; // Built-in executed
}
return 0; // Not a built-in
}

```

// Execute a single command

```

void execute_single(char **args) {
    if (execute_builtin(args)) {
        return;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("shell: fork failed");
        return;
    }

    if (pid == 0) {
        // Child process
        execvp(args[0], args);
        perror("shell: command not found");
        exit(1);
    } else {
        // Parent process
        wait(NULL);
    }
}

```

// Execute piped commands

```

void execute_pipe(char **cmd_strings, int num_cmds) {
    int pipes[MAX_CMDS - 1][2];
    pid_t pids[MAX_CMDS];

```

```

// Create all necessary pipes
for (int i = 0; i < num_cmds - 1; i++) {
    if (pipe(pipes[i]) < 0) {
        perror("shell: pipe failed");
        return;
    }
}

// Fork and execute each command
for (int i = 0; i < num_cmds; i++) {
    char *args[MAX_ARGS];
    parse_command(cmd_strings[i], args);

    if (args[0] == NULL) continue;

    pids[i] = fork();
    if (pids[i] < 0) {
        perror("shell: fork failed");
        return;
    }

    if (pids[i] == 0) {
        // Child process

        // Set up input from previous pipe
        if (i > 0) {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }

        // Set up output to next pipe
        if (i < num_cmds - 1) {
            dup2(pipes[i][1], STDOUT_FILENO);
        }

        // Close all pipe ends
        for (int j = 0; j < num_cmds - 1; j++) {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        // Execute built-in or external command
        if (execute_builtin(args)) {
            exit(0);
        }
    }
}

```

```

        execvp(args[0], args);
        perror("shell: command not found");
        exit(1);
    }
}

// Parent: close all pipe ends
for (int i = 0; i < num_cmds - 1; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
}

// Wait for all children
for (int i = 0; i < num_cmds; i++) {
    wait(NULL);
}
}

int main() {
    char line[MAX_LINE];
    char *cmds[MAX_CMDS];

    while (1) {
        // Print prompt
        printf("shell> ");
        fflush(stdout);

        // Read input
        if (fgets(line, sizeof(line), stdin) == NULL) {
            printf("\n");
            break;
        }

        // Skip empty lines
        if (line[0] == '\n' || line[0] == '\0') {
            continue;
        }

        // Parse line into commands
        int num_cmds = parse_line(line, cmds);

        if (num_cmds == 0) {
            continue;
        }
    }
}

```

```
}

// Execute
if (num_cmds == 1) {
    char *args[MAX_ARGS];
    parse_command(cmds[0], args);
    if (args[0] != NULL) {
        execute_single(args);
    }
} else {
    execute_pipe(cmds, num_cmds);
}

return 0;
}
```